HUNT-SAGE · THREAT HUNTING CHEAT SHEET

Network & DNS Hunting (Zeek)

Zeek 6/7 logs and the fields that matter, conn_state and history decoding, JA3/JA4 fingerprinting, DNS and beacon checklists, zeek-cut/jq one-liners, east-west ports and PCAP triage.

github.com/iamjammie/hunt-sage · iamjammie.github.io/hunt-sage · Built 2026-11-03

Zeek turns packets into typed, joinable logs. Every connection gets a `uid`, every file a `fuid`, and (with the `community-id` policy loaded) a `community_id` that matches Suricata, EDR and cloud flow logs. Join on those, not on IPs and timestamps. Methods for the timing analysis below are in the C2 beaconing playbook.

Zeek logs that matter

Log	Key fields	Hunting use
`conn.log`	`uid`, `id.orig_h/p`, `id.resp_h/p`, `proto`, `service`, `duration`, `orig_bytes`, `resp_bytes`, `conn_state`, `history`, `orig_pkts`, `resp_pkts`, `local_orig`, `community_id`	Beaconing, scanning, exfil volume, long sessions, first-seen pairs; the spine every other log joins to
`dns.log`	`query`, `qtype_name`, `rcode_name`, `answers`, `TTLs`, `rejected`, `rtt`, `trans_id`	Tunnelling, DGA, rare domains, resolver bypass, sinkhole answers
`http.log`	`method`, `host`, `uri`, `user_agent`, `status_code`, `request_body_len`, `response_body_len`, `resp_mime_types`, `orig_fuids` / `resp_fuids`, `proxied`, `referrer`	Rare user agents, downloads, C2 over HTTP, web shells (POSTs to a static file), long URIs
`ssl.log`	`server_name` (SNI), `version`, `cipher`, `established`, `ssl_history` (6.0+), `cert_chain_fps`, `sni_matches_cert` (6.0+), `validation_status`, `ja3` / `ja4` with packages	C2 over TLS, self-signed or mismatched certs, rare client fingerprints, DoH
`x509.log`	`fingerprint` (6.0+), `certificate.subject`, `certificate.issuer`, `certificate.not_valid_before/after`, `san.dns`, `certificate.key_length`	Certificate pivots: young certs, default subjects (`CN=localhost`), reused keys
`files.log`	`fuid`, `uid`, `id.orig_h`, `id.resp_h`, `source`, `mime_type`, `filename`, `md5` / `sha1` / `sha256` (with `hash-all-files`), `extracted`, `total_bytes`	Payload delivery, exfil of archives, executables over unusual protocols
`notice.log`	`note` (`Scan::Address_Scan`, `SSL::Invalid_Server_Cert`, `SSH::Password_Guessing`, `Signatures::Sensitive_Signature`), `msg`, `sub`, `actions`	Leads from Zeek's own detections and loaded packages
`weird.log`	`name` (`bad_TCP_checksum`, `data_before_established`, `DNS_RR_unknown_type`, `unknown_protocol`), `addl`	Protocol anomalies: tunnelling, evasion, broken implants
`smb_files.log`	`action` (`SMB::FILE_OPEN/WRITE/DELETE/RENAME`), `path`, `name`, `size`	Lateral movement payload drops, ransomware mass writes, staging on shares
`smb_mapping.log`	`path` (`\\host\ADMIN$`, `C$`, `IPC$`), `share_type`, `service`	Admin share use, share fan-out from one host
`kerberos.log`	`request_type` (AS/TGS), `client`, `service`, `success`, `error_msg`, `cipher`, `forwardable`, `renewable`, `till`	Kerberoasting (RC4 TGS), AS-REP roasting, ticket lifetime oddities, pre-auth failures as a spray signal
`ntlm.log`	`username`, `hostname`, `domainname`, `server_nb_computer_name`, `server_dns_computer_name`, `success`	Pass-the-hash, relay (client host name does not match source IP), NTLM where Kerberos is expected
`dce_rpc.log`	`endpoint`, `operation`, `rtt`, `named_pipe`	DCSync (`drsuapi` `DRSGetNCChanges`), remote service (`svcctl` `CreateServiceW`), tasks (`atsvc`, `ITaskSchedulerService`), share enum (`srvsvc` `NetrShareEnum`), `samr` / `lsarpc` recon

Log	Key fields	Hunting use
`rdp.log`	`cookie` (client username), `result`, `security_protocol`, `client_name`, `client_build`, `cert_type`	RDP from workstations to workstations, new client names, NLA disabled
`ssh.log`	`direction` (INBOUND/OUTBOUND), `auth_success`, `auth_attempts`, `client`, `server`, `host_key`, `hassh` / `hasshServer` with the package	Brute force, outbound SSH from servers, unusual client software, tunnels
`ftp.log`, `smtp.log`	`user`, `command`, `arg`, `file_size`; `mailfrom`, `rcptto`, `subject`, `fuids`, `tls`	Legacy exfil; phishing delivery and forwarding
`dhcp.log`, `software.log`, `known_services.log`, `known_hosts.log`	`assigned_addr`, `host_name`, `client_fqdn`; `software_type`, `unparsed_version`; `port_num`, `service`	Asset context: who had the IP, what runs where, a new listening service on an old host
`ldap.log`, `ldap_search.log` (6.1+)	`operation`, `object`, `filter`, `scope`, `attributes`, `result`	AD recon from the wire (`(servicePrincipalName=*)`, `(objectClass=*)` with subtree scope)
`quic.log` (6.1+), `tunnel.log`, `analyzer.log`	`server_name`, `client_initial_dcid`; `tunnel_type`; analyzer violations	QUIC/HTTP3 C2 and DoH over QUIC, encapsulation, protocol mismatches on standard ports

`conn_state` and `history`

State	Meaning	Letter	`history` meaning (upper = responder)
`S0`	SYN seen, no reply: scan, blocked, or dead host	`s` / `S`	SYN
`S1`	Established, never closed (still open or capture ended)	`h` / `H`	SYN-ACK
`SF`	Normal establish and teardown	`a` / `A`	ACK
`REJ`	SYN then RST: rejected	`d` / `D`	payload data
`S2` / `S3`	Established, closed by one side only	`f` / `F`	FIN
`RSTO` / `RSTR`	Reset by originator / responder after establishment	`r` / `R`	RST
`RSTOS0` / `RSTRH`	Originator SYN then its RST; responder SYN-ACK then its RST	`c` / `C`	bad checksum
`SH` / `SHR`	Half-open: SYN then FIN with no reply (`SHR` from responder)	`t` / `T`	retransmission
`OTH`	No SYN seen; mid-stream (long-lived, asymmetric capture, capture started late)	`g` / `G`, `w` / `W`, `i` / `I`, `q` / `Q`, `^`	content gap, zero window, inconsistent packet, multi-flag, direction flip

`history` repeats a letter when the event recurs; `ShADadFf` is a clean exchange, `S` alone is a scan, `ShAD` with `S1` and a large `duration` is a session worth reading, and `^` means Zeek guessed the direction.

TLS fingerprinting: JA3, JA3S, JA4

Fingerprint	Built from	Use and limits
JA3 (client)	TLS version, ciphers, extensions, curves, point formats, MD5	Fast stacking of client software; brittle (extension order randomised by browsers since 2023, so one browser yields many JA3s)
JA3S (server)	Version, cipher, extensions of the ServerHello	Pair with JA3: a known C2 server answers a known client the same way every time
JA4 (client)	`t13d1516h2_<cipher hash>_<extension hash>`: protocol, version, SNI present, cipher and extension counts, ALPN, then sorted hashes	Order-independent, human-readable prefix, far more stable than JA3; the FoxIO `zeek-ja4` package adds `ja4`, `ja4s`, `ja4h` (HTTP), `ja4x` (x509), `ja4t` / `ja4ts` (TCP), `ja4l` (latency)
JA4S / JA4X	Server hello; certificate issuer and subject RDN order	Server-side and certificate-side stacking; `ja4x` finds self-made certs that copy real subjects in the wrong order
HASSH	SSH KEX, ciphers, MACs, compression	Client/server SSH software fingerprint in `ssh.log`

TLS 1.3 encrypts the certificate, so `ssl.log` has no `cert_chain_fps` or `x509.log` rows for most modern sessions: pivot on SNI, JA4, JA4S and `ssl_history` instead. **Encrypted Client Hello** hides the SNI too (rare in enterprises; an `ech` extension or an empty `server_name` to a CDN is itself a lead). Install fingerprinting with `zkg install zeek/salesforce/ja3`, `zkg install foxio/ja4` and `zkg install zeek/salesforce/hassh`; load `policy/protocols/conn/community-id-logging` and `policy/frameworks/files/hash-all-files` in `local.zeek`.

DNS anomaly checklist

- **Length and entropy**: queries over 50 characters, labels over 30, high Shannon entropy per label, many unique subdomains of one parent (tunnel or DGA); compute entropy in the SIEM, not in Zeek
- **Record types**: unusual volume of `TXT`, `NULL`, `CNAME`, `MX` or `ANY` from one client; `TXT` answers carrying base64
- **NXDOMAIN bursts**: high `rcode_name == NXDOMAIN` ratio per client (DGA); a sudden drop after a burst is a successful rendezvous
- **Resolver bypass**: UDP/TCP 53 to anything but your resolvers; TCP 53 from workstations; `dns.log` entries whose `id.resp_h` is external
- **DoH / DoT**: TCP 853 (DoT); TLS to known DoH endpoints (`dns.google`, `cloudflare-dns.com`, `dns.quad9.net`, `doh.opendns.com`) by SNI; QUIC on 443 to resolver IPs; a host with HTTPS traffic but almost no `dns.log` rows
- **Young and rare**: newly registered domains (under 30 days; enrich with a registration-date feed), rare TLDs for your estate (`.top`, `.xyz`, `.click`, `.icu` are overrepresented, lead only), punycode (`xn--`) look-alikes of your own brands
- **Answers**: low TTLs with many changing A records (fast flux), answers in RFC 1918 or loopback space for external names (rebinding, sinkholes), `0.0.0.0`
- **Services**: dynamic DNS and tunnel providers (`duckdns`, `no-ip`, `ngrok`, `trycloudflare`, `serveo`, `localtunnel`), paste and file-sharing domains from servers
- **Timing**: the same name queried at a fixed interval from one host, see the beacon section below

Beacon and C2 detection methods

Method	What to measure	Notes
Interval regularity	Per (`id.orig_h`, `id.resp_h`, `id.resp_p`): sort by `ts`, take deltas, then count, mean, standard deviation and coefficient of variation	Low CoV with many connections is a beacon; expect jitter of 10-50 %, so use percentiles or MAD rather than exact intervals
Jitter shape	Histogram of deltas: implants produce a tight band, humans produce a long tail	Cobalt Strike default `sleep 60` with `jitter 20` gives deltas in a 48-72 s band
Byte symmetry	`orig_bytes` and `resp_bytes` nearly constant per connection; tiny request, tiny response, hundreds of times a day	Also flag the opposite: one huge `orig_bytes` to a never-seen host (exfil)
Long sessions	`duration` over an hour, `conn_state` `S1` / `OTH` to external IPs	Reverse shells, persistent WebSockets, VPN-like tunnels
Rarity	Destinations contacted by one or two hosts in the estate, JA4s seen on one host, user agents seen once	Stack by day over 30 days; the long tail is the hunt
Protocol mismatch	`service` not matching `id.resp_p` (HTTP on 8443, SSH on 443, `-` on 443 with data) and `weird.log` names	Zeek's dynamic protocol detection does the work
Scoring tools	**RITA** (Active Countermeasures, Go rewrite) and **AC-Hunter** score beacons, long connections, DNS tunnelling and threat-intel hits from Zeek logs; **BZAR** maps `dce_rpc` / `smb` to ATT&CK lateral-movement techniques; **zeek-sniffpass** flags clear-text credentials	Point them at a day of logs first; they are fast

zeek-cut and jq one-liners

```
# Human-readable times, selected columns (TSV logs)
cat conn.log | zeek-cut -d ts id.orig_h id.resp_h id.resp_p service duration orig_bytes resp_bytes conn_state | head
# Top talkers by bytes out (candidate exfil)
cat conn.log | zeek-cut id.orig_h id.resp_h orig_bytes | awk '{b[$1" "$2]+=$3} END{for(k in b)print b[k],k}' | sort
-rn | head
# Rare external destinations: pairs seen from only one source host
cat conn.log | zeek-cut id.orig_h id.resp_h local_resp | awk '$3=="F"' | sort -u | awk '{n[$2]++} END{for(d in n) if
(n[d]==1) print d}' | head -50
# First-seen (orig, resp, port) pairs today versus a 30-day baseline file
cat conn.log | zeek-cut id.orig_h id.resp_h id.resp_p | sort -u > today.txt; comm -23 today.txt baseline_pairs.txt |
head
# Long connections (> 1 hour) still open or mid-stream
cat conn.log | zeek-cut -d ts id.orig_h id.resp_h id.resp_p duration conn_state | awk '$5>3600' | sort -k5 -rn | hea
d
# Beacon candidates: connection count and interval CoV per pair (needs at least 20 connections)
cat conn.log | zeek-cut ts id.orig_h id.resp_h id.resp_p | sort -k2,4 -k1n | awk '{k=$2" "$3" "$4; if(k==p){d=$1-t;
n[k]++; s[k]+=d; ss[k]+=d*d} p=k; t=$1} END{for(k in n) if(n[k]>=20){m=s[k]/n[k]; v=ss[k]/n[k]-m*m; printf "%d %.1f
%.2f %s\n", n[k], m, (v>0?sqrt(v)/m:0), k}}' | sort -k3n | head
# SMB share fan-out: one host mapping many admin shares
cat smb_mapping.log | zeek-cut id.orig_h path | grep -Ei 'admin\$|c\$' | sort -u | cut -f1 | uniq -c | sort -rn | he
ad
# Kerberos: RC4 service-ticket requests (Kerberoasting signal) and pre-auth failures per client
cat kerberos.log | zeek-cut id.orig_h client service request_type cipher | awk '$4=="TGS" && $5 ~ /rc4/' | sort | un
iq -c | sort -rn | head
cat kerberos.log | zeek-cut id.orig_h error_msg | grep -i preauth | cut -f1 | sort | uniq -c | sort -rn | head
# NTLM from a host whose client name does not match its IP (relay or pass-the-hash lead)
cat ntlm.log | zeek-cut id.orig_h hostname username success | sort -u | head -40
# DNS: longest queries, NXDOMAIN storms, TXT-heavy clients
cat dns.log | zeek-cut query | awk '{print length($0), $0}' | sort -rn | head
cat dns.log | zeek-cut id.orig_h rcode_name | grep NXDOMAIN | cut -f1 | sort | uniq -c | sort -rn | head
cat dns.log | zeek-cut id.orig_h qtype_name | awk '$2=="TXT"' | cut -f1 | sort | uniq -c | sort -rn | head
# TLS: rare JA4s, self-signed or mismatched certs, SNI-less sessions
cat ssl.log | zeek-cut ja4 | sort | uniq -c | sort -n | head -30
cat ssl.log | zeek-cut server_name validation_status sni_matches_cert | grep -Ev 'ok\s+T' | sort | uniq -c | sort -r
n | head
# JSON logs (LogAscii::use_json=T) with jq
jq -r 'select(.["id.resp_p"]==445) | [.["id.orig_h"], .["id.resp_h"], .conn_state] | @tsv' conn.log | sort | uniq -c
| sort -rn | head
jq -r 'select(.query|length>50) | [.["id.orig_h"], .query] | @tsv' dns.log | head
# Pivot a connection to its files and HTTP rows by uid
grep -F "$UID" files.log http.log ssl.log | zeek-cut -d ts uid id.resp_h
```

Ports of interest (east-west)

Port	Service	What abuse looks like
22	SSH	Server-to-server pivoting, `-R` / `-D` tunnels, outbound SSH from servers that never used it
88, 464	Kerberos, kpasswd	RC4 TGS bursts (roasting), AS-REQ without pre-auth, password changes from odd hosts
135, 49152-65535	RPC endpoint mapper and dynamic ports	Remote WMI, service and task creation, DCOM; `dce_rpc.log` names the operation
139, 445	SMB	PsExec-style service installs, `ADMIN$` / `C$` writes, mass file writes (ransomware), share enumeration fan-out
389, 636, 3268, 3269	LDAP, LDAPS, Global Catalog	AD recon (`ldap_search.log` filters), LDAP relay, bulk queries from workstations
3389	RDP	Workstation-to-workstation RDP, new `client_name` s, NLA disabled, RDP tunnelled over SSH
5985, 5986	WinRM	PowerShell remoting from non-admin hosts
1433, 1521, 3306, 5432, 27017, 6379	Databases, Redis	Bulk reads, `xp_cmdshell` , unauthenticated Redis as a foothold
53, 853, 443/udp	DNS, DoT, QUIC	Tunnelling, resolver bypass, DoH over QUIC
80, 443, 8080, 8443	HTTP(S) and proxies	C2 blends here; hunt by rarity, timing and fingerprint, not by port
4444, 1337, 31337, 50050	Default implant/listener ports	Lead only; mature C2 picks 443
21, 20, 69, 2049	FTP, TFTP, NFS	Legacy exfil and staging; NFS mounts from unexpected clients
161, 162, 514, 5601, 9200	SNMP, syslog, Kibana, Elasticsearch	Recon of monitoring and log stores; log-destruction attempts

Port	Service	What abuse looks like
2375, 2376, 6443, 10250	Docker API, Kubernetes API, kubelet	Unauthenticated container runtime access; see the Kubernetes playbook

PCAP quick triage

```
capinfos cap.pcap                                                    # duration, size, packet count
zeek -C -r cap.pcap local                                            # Zeek logs from a pcap (-C ignores checksum
s)
suricata -r cap.pcap -l out/                                         # IDS alerts into eve.json
tshark -r cap.pcap -q -z conv,tcp | sort -k10 -rn | head -30         # conversations by bytes
tshark -r cap.pcap -q -z endpoints,ip | head -30
tshark -r cap.pcap -Y 'http.request' -T fields -e ip.src -e http.host -e http.request.uri -e http.user_agent | sort
| uniq -c | sort -rn | head
tshark -r cap.pcap -Y 'dns.flags.response == 0' -T fields -e ip.src -e dns.qry.name -e dns.qry.type | sort | uniq -c
| sort -rn | head
tshark -r cap.pcap -Y 'tls.handshake.type == 1' -T fields -e ip.src -e ip.dst -e tls.handshake.extensions_server_nam
e | sort | uniq -c | sort -rn | head
tshark -r cap.pcap -Y 'tcp.flags.syn == 1 && tcp.flags.ack == 0' -T fields -e ip.src -e ip.dst -e tcp.dstport | sort
| uniq -c | sort -rn | head   # SYN scans
tcpdump -nn -r cap.pcap 'port 445 and tcp[tcpflags] & tcp-syn != 0' | head  # SMB connection attempts
tshark -r cap.pcap --export-objects http,objects/                     # carve HTTP downloads for hashing
```

Ship Zeek to your SIEM

Platform	Notes
Splunk	Splunk TA for Zeek (`sourcetype=zeek:conn:json` , `zeek:dns:json` , ...) or the Corelight App (`corelight_conn`); CIM mapping: `conn.log` to `Network_Traffic` (`src` , `dest` , `dest_port` , `bytes_in` / `bytes_out` , `transport`), `dns.log` to `Network_Resolution` (`query` , `reply_code` , `answer`), `http.log` to `Web` (`url` , `http_user_agent` , `status`), `ssl.log` to `Certificates` . Field names keep the dotted Zeek form (`id.orig_h`)
Microsoft Sentinel	Corelight data connector (`Corelight_v2_conn_CL` and siblings) or a custom log via the Azure Monitor Agent; ASIM `_Im_NetworkSession` and `_Im_Dns` parsers give a vendor-neutral schema (`SrcIpAddr` , `DstIpAddr` , `DstPortNumber` , `DnsQuery`)
Elastic	Filebeat `zeek` module to ECS (`source.ip` , `destination.ip` , `network.community_id` , `zeek.connection.state`), which joins cleanly with Suricata's module
Join keys	`uid` across Zeek logs, `fuid` to `files.log` , `community_id` across Zeek, Suricata, Corelight, AWS/Azure flow logs and EDR network events

```
sourcetype=zeek:conn:json earliest=-24h local_orig=true local_resp=false
| eval src='id.orig_h', dest='id.resp_h', dport='id.resp_p'
| sort 0 src dest dport _time
| streamstats current=f last(_time) as prev by src dest dport
| eval delta=_time-prev
| stats count avg(delta) as avg_delta stdev(delta) as sd_delta sum(orig_bytes) as out_bytes by src dest dport
| where count>=20 AND avg_delta>10 AND sd_delta/avg_delta<0.3
| sort -count
```

```
Corelight_v2_conn_CL
| where TimeGenerated > ago(1d) and local_orig_b == true and local_resp_b == false
| sort by id_orig_h_s, id_resp_h_s, id_resp_p_d, TimeGenerated asc
| extend Delta = datetime_diff("second", TimeGenerated, prev(TimeGenerated)), SamePair = (id_orig_h_s == prev(id_ori
g_h_s) and id_resp_h_s == prev(id_resp_h_s) and id_resp_p_d == prev(id_resp_p_d))
| where SamePair
| summarize Count = count(), AvgDelta = avg(Delta), SdDelta = stdev(Delta), OutBytes = sum(orig_bytes_d) by id_orig_
h_s, id_resp_h_s, id_resp_p_d
| where Count >= 20 and AvgDelta > 10 and SdDelta / AvgDelta < 0.3
| order by Count desc
```

Column names depend on the connector version; adapt them. Related: C2 beaconing, data exfiltration, lateral movement, the query index and practice datasets with public Zeek logs.